Apellidos:
Nombre:
Asignatura: Programación de Concurrencia y Distribuida
Grupo: **Subgrupo:**
Convocatoria:
Año académico: 2025–2026
Profesor:

MEMORIA DE PRÁCTICAS

Título del Proyecto

Índice

1. Ejercicio 1: ReentrantLock **2**
1.1. Recursos no compartibles 2
1.2. Condiciones de sincronización 2
1.3. Pseudocódigo comentado 2
1.4. Cuestiones planteadas en los ejercicios 4
1.5. Código Java de los ejercicios comentado 5

2. Ejercicio 2: Semáforos **6**
2.1. Recursos no compartibles 6
2.2. Condiciones de sincronización 6
2.3. Pseudocódigo comentado 6
2.4. Cuestiones planteadas en los ejercicios 6
2.5. Código Java de los ejercicios comentado 6

3. Ejercicio 3: Monitores **7**
3.1. Recursos no compartibles 7
3.2. Condiciones de sincronización 7
3.3. Pseudocódigo comentado 7
3.4. Cuestiones planteadas en los ejercicios 7
3.5. Código Java de los ejercicios comentado 7

4. Ejercicio 4: Paso de Mensajes Asíncrono **8**
4.1. Recursos no compartibles 8
4.2. Condiciones de sincronización 8
4.3. Pseudocódigo comentado 8
4.4. Cuestiones planteadas en los ejercicios 8
4.5. Código Java de los ejercicios comentado 8

1. Ejercicio 1: ReentrantLock

1.1. Recursos no compartibles

1.2. Condiciones de sincronización

1.3. Pseudocódigo comentado

```
Algoritmo: Clase Main
// Variables globales
Datos:
cerrojoPantalla : Cerrojo
NUM_OPERACIONES : Entero
LONGITUD_FILAS_Y_COLUMNAS : Entero
NUM_0_9 : Entero
OPCION_A : Caracter
OPCION_R : Caracter
// Inicialización de variables globales
NUM_OPERACIONES ← 10
LONGITUD_FILAS_Y_COLUMNAS ← 3
NUM_0_9 ← 10
OPCION_A ←' a'
OPCION_R ←' r'
Función main()
    initial (cerrojoPantalla, 1)
    // Creación y arraque de 2 hilos
    // SumaMatriz y CuadradoMatriz

fin
```

```
Algoritmo: Clase SumaMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Matriz[][]
impactoAcumulado : Matriz[][]
metricasDeEntrenamientoCadena : Cadena
impactoAcumuladoCadena : Cadena

Función run()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular suma de matrices
        // Guardar resultado en impactoAcumulado

        metricasDeEntrenamientoCadena ← matrizToString
         (metricasDeEntrenamiento, OPCION_A)
        impactoAcumuladoCadena ← matrizToString
         (impactoAcumulado, OPCION_R)
        wait(cerrojoPantalla)
        // Impresión en pantalla de las matrices
        signal(cerrojoPantalla)
    fin
fin
```

```
Algoritmo: Clase SumaMatriz
// Variables de instancia
Datos:
metricasDeEntrenamiento : Matriz[][]
impactoAcumulado : Matriz[][]
metricasDeEntrenamientoCadena : Cadena
impactoAcumuladoCadena : Cadena

Función run()
    para i ← 1 a NUM_OPERACIONES hacer
        generarMatriz (metricasDeEntrenamiento)
        // Calcular suma de matrices
        // Guardar resultado en impactoAcumulado

        metricasDeEntrenamientoCadena ← matrizToString
         (metricasDeEntrenamiento, OPCION_A)
        impactoAcumuladoCadena ← matrizToString
         (impactoAcumulado, OPCION_R)
        wait(cerrojoPantalla)
        // Impresión en pantalla de las matrices
        signal(cerrojoPantalla)
    fin
fin
```

1.4. Cuestiones planteadas en los ejercicios

a) **Si no usas sincronización, ¿cómo podría ser la salida por pantalla?**

Como regla general, si no usas sincronización, la salida por pantalla es caótica e impredecible. En este caso, dado que ambos hilos (SumaMatriz y CuadradoMatriz) están accediendo a la pantalla para imprimir sus resultados, sin sincronización podrían intercalarse las impresiones de ambos hilos. Esto podría resultar en una salida mezclada donde partes de la matriz de SumaMatriz se imprimen junto con partes de la matriz de CuadradoMatriz, haciendo que sea difícil o imposible entender los resultados.

En el caso particular de java cada System.out.println() es una operación atómica, lo que significa que cada llamada a esta función se ejecutará completamente sin ser interrumpida por otro hilo. Sin embargo, esto no garantiza que las impresiones de ambos hilos no se mezclen, ya que cada hilo podría imprimir varias líneas de la matriz antes de que el otro hilo tenga la oportunidad de imprimir. Por lo tanto, aunque cada línea individual se imprima

de manera atómica, la secuencia completa de impresiones podría resultar en una salida caótica si no se sincroniza adecuadamente.

1.5. Código Java de los ejercicios comentado

Clase Main

Clase SumaMatriz

Clase CuadradoMatriz

2. Ejercicio 2: Semáforos

2.1. Recursos no compartibles

2.2. Condiciones de sincronización

2.3. Pseudocódigo comentado

2.4. Cuestiones planteadas en los ejercicios

Según las condiciones del problema:

a) **(0.5 puntos sobre 10) Si consideramos un panel solamente ¿cómo es el acceso al mismo?** Si consideramos los tres paneles ¿cómo es el acceso? Explica ambas situaciones.

b) **(0.5 puntos sobre 10) ¿Qué tipo de semáforos has usado para resolver el problema (binarios, generales)?**

2.5. Código Java de los ejercicios comentado

3. Ejercicio 3: Monitores

3.1. Recursos no compartibles

3.2. Condiciones de sincronización

3.3. Pseudocódigo comentado

3.4. Cuestiones planteadas en los ejercicios

a) **(0.5 puntos) Indica si la acción de entrenar es concurrente o en exclusión mutua y justifica.**

b) **(0.75 puntos) ¿Qué tipo de monitor Java has usado? Justifica.**

c) **(0.5 puntos) En el monitor, ¿has usado notify/signal o notifyAll/signalAll? Justifica**

d) **(0.75 puntos) ¿Cómo se ha resuelto la exclusión mutua de la pantalla en este problema**

3.5. Código Java de los ejercicios comentado

Clase CuadradoMatriz

4. Ejercicio 4: Paso de Mensajes Asíncrono

4.1. Recursos no compartibles

4.2. Condiciones de sincronización

4.3. Pseudocódigo comentado

4.4. Cuestiones planteadas en los ejercicios

a) **(0.5 puntos sobre 10) ¿Se pueden usar simultáneamente los dos tornos? Justifica la respuesta**

b) **(0.5 puntos sobre 10) ¿Cómo has resuelto la exclusión mutua de la pantalla?**

4.5. Código Java de los ejercicios comentado

Clase CuadradoMatriz